

PROJECT CONTEXT

Project Title

Business-Impact Aware Incident Prioritization for Microservice Systems using Dependency Graph Intelligence

Goal

This project is part of my final-year Software Engineering research.

The end goal is:

1. Publish a research paper.
2. Build a working prototype.
3. Demonstrate novelty over existing AIOps systems.
4. Solve a practical problem faced by DevOps/SRE teams.

The project must contain both

- a novel algorithm/framework
 - an actual implementation.
-

Problem Statement

Modern microservice systems generate thousands of alerts every day.

Existing AIOps systems can detect incidents, classify failures, and locate root causes.

However they still leave engineers with an important unanswered question:

Which incident should be fixed first?

Current systems usually prioritize incidents using static severity levels or manually assigned business rules.

This ignores

- dependency propagation
- business workflows
- downstream impact
- service importance
- user impact
- cascading failures

As a result,

two incidents having the same technical severity may have completely different business impact.

Current Research Direction

The research focuses on **Business-aware Incident Prioritization** rather than anomaly detection or root cause localization.

The system should accept detected incidents and intelligently prioritize them using graph-based reasoning.

Scope

The project starts **after** incident detection.

Incident Detection

↓

Failure Classification

↓

Root Cause Localization

↓

Business Impact Analysis

↓

Priority Optimization

↓

Final Incident Ranking

Existing Work

Current systems already perform

- Anomaly Detection (AD)
- Failure Triage (FT)
- Root Cause Localization (RCL)

Examples include

- TrioXpert
- ART
- DiagFusion
- Eadro

These systems are **NOT** my contribution.

They act as upstream modules.

My work begins after they produce incident candidates.

Overall Architecture

Monitoring Stack

↓

Metrics

Logs

Traces

↓

Data Collection



Preprocessing



Incident Detection Engine



Failure Classification



Root Cause Localization



Business Dependency Graph Construction



Graph Feature Extraction



Business Impact Analysis



Priority Optimization



Incident Ranking Dashboard

Research Contribution

The novelty should NOT be another anomaly detection model.

The novelty should lie in intelligent incident prioritization.

Possible contributions include

- Business Dependency Graph
- Business Knowledge Graph

- Graph Neural Network
 - Graph Attention Network
 - Graph-based propagation algorithm
 - Dependency-aware optimization
 - Business-aware ranking algorithm
-

Proposed Business Dependency Graph

Instead of representing only service-to-service calls, the graph should model business knowledge.

Nodes may include

- Microservices
- APIs
- Databases
- Message Queues
- Caches
- Deployments
- Business Capabilities
- User-facing Features
- Teams
- Infrastructure Components

Edges may represent

- API Calls
- Data Flow
- Deployment Dependency
- Business Workflow

- Runtime Dependency
 - Shared Database
 - Shared Infrastructure
-

Each Node May Store

Service Name

Service Criticality

Owner Team

Average Latency

Error Rate

Request Volume

Deployment Region

SLA

SLO

Availability

Business Function

Revenue Contribution

Historical Incidents

Recovery Cost

MTTR

Deployment Frequency

Failure Frequency

Dependency Count

User Traffic

Incident Input

Each incident contains

- Incident ID
- Timestamp
- Affected Service
- Root Cause
- Failure Type
- Technical Severity
- Error Metrics
- Logs
- Traces
- Affected APIs
- Deployment Information

Business Impact Engine

For every incident compute

- Affected Services
- Affected APIs
- Affected Users
- Affected Transactions
- Propagation Depth
- Dependency Centrality
- Revenue Impact
- Business Capability Impact
- Customer Impact
- SLA Risk
- MTTR Risk

Recovery Cost

Historical Importance

Priority Score

Instead of severity,

calculate

Priority Score =

Business Impact

-

Technical Severity

-

Propagation Risk

-

Critical Path Importance

-

Dependency Centrality

-

Historical Failure Importance

-

Recovery Cost

-

SLA Violation Risk

Weights may be learned automatically instead of manually assigned.

Graph Algorithms Worth Investigating

PageRank
Personalized PageRank
Graph Neural Networks
Graph Attention Networks
GraphSAGE
Message Passing Networks
Graph Transformers
Betweenness Centrality
Closeness Centrality
Eigenvector Centrality
Shortest Path
Dependency Traversal
Topological Sorting
Community Detection
Influence Maximization

Possible Novel Optimization Techniques

Instead of static rules,
learn priority using
Graph Neural Networks
Graph Attention Networks
Learning-to-Rank
Pairwise Ranking
Reinforcement Learning
Graph Embeddings
Node Importance Prediction

Possible Inputs

Metrics

CPU

Memory

Disk

Latency

Response Time

Request Rate

Availability

Logs

Errors

Exceptions

Timeouts

Warnings

Traces

Service Calls

Execution Chains

Latency

Span Status

Deployment Metadata

Service Versions

Containers

Pods

Nodes

Namespaces

Business Metadata

Revenue

Customer Importance

Business Workflow

Feature Ownership

SLA

Team

Expected Outputs

Incident Priority

Business Impact Score

Propagation Path

Affected Services

Critical Path

Reasoning

Confidence Score

Suggested Resolution Order

Evaluation

Possible datasets

Sock Shop

Train Ticket

DeathStarBench

OpenTelemetry Demo

Online Boutique

Google Microservices Demo
Synthetic Kubernetes Clusters
Real Production Logs (if available)

Evaluation Metrics

Priority Ranking Accuracy

Precision

Recall

F1

MRR

NDCG

MAP

Top-K Accuracy

Average Precision

Latency

Inference Time

Memory Usage

Scalability

Baselines

Compare against

Severity-based Prioritization

Rule-based Prioritization

Static Dependency Ranking

ART

TrioXpert

Existing AIOps Systems

PagerDuty Policies

ServiceNow Prioritization

Expected Novelty

The novelty should not be

- ✗ Another anomaly detector
- ✗ Another root cause localizer
- ✗ Another log parser

Instead it should be

- ✓ New graph representation
 - ✓ New dependency-aware prioritization algorithm
 - ✓ Business-aware optimization framework
 - ✓ Intelligent incident ranking
-

Research Questions

RQ1

Can dependency-aware graph reasoning improve incident prioritization?

RQ2

Can business-aware features outperform static severity?

RQ3

Which graph representation best captures business impact?

RQ4

Can graph learning reduce incident response time?

RQ5

Can explainable graph reasoning improve engineer trust?

Expected Deliverables

- Research paper
 - System architecture
 - Dependency graph construction module
 - Business impact analyzer
 - Priority optimization algorithm
 - Dashboard
 - Experimental evaluation
 - Ablation study
 - Comparison with baselines
 - Open-source implementation
-

Expectations from the AI Assistant

When helping with this project:

- Think like a research collaborator, not just a coding assistant.
- Challenge assumptions and point out when an idea already exists in the literature.
- Suggest novel research directions grounded in recent work.
- Prioritize publishable contributions over engineering complexity.
- Help identify research gaps, relevant papers, datasets, baselines, and evaluation strategies.
- Distinguish clearly between existing work and potential contributions.
- When proposing an architecture or algorithm, explain why it is novel, how it differs from prior work, and what experiments would be needed to validate it.
- If an idea seems weak or already solved, say so directly and suggest stronger alternatives.